SRI CHANDRASEKHADRENDRA SARASWATHI VISWA MAHAVIDYALAYA

UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1G56) ENATHUR, KANCHIPURAM - 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Name : P. Sri Manaswini

Reg No : 11249A356

Class : II B.E(CSE)

Course Code :

Course Name :. Data Science Toolkit Lab.

SRI CHANDRASEKHADRENDRA SARASWATHI VISWA MAHAVIDYALAYA

UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1G56) ENATHUR, KANCHIPURAM - 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this is the Bonafide record of work done by

Ms. P. Sri Manaswini

with Reg. No 11249A356 done by of II-B.E.(CSE) in DATA SCIENCE TOOLKITfor Lab during the academic year 2025 -2026.

Station : Enathur

Date :

Staff-in-charge

Head of the department

Submitted for the Practical examination held on_____

Examine-1

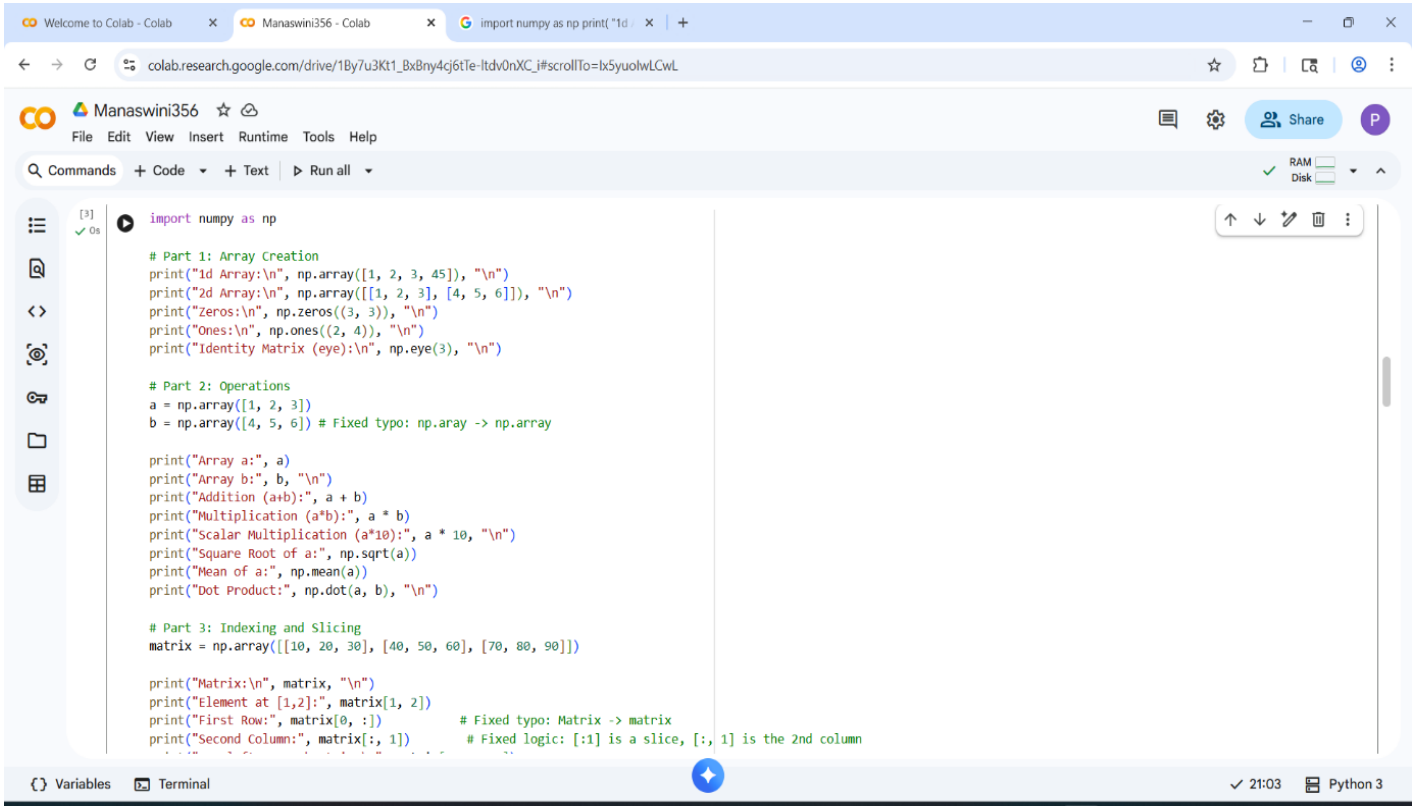
Examine

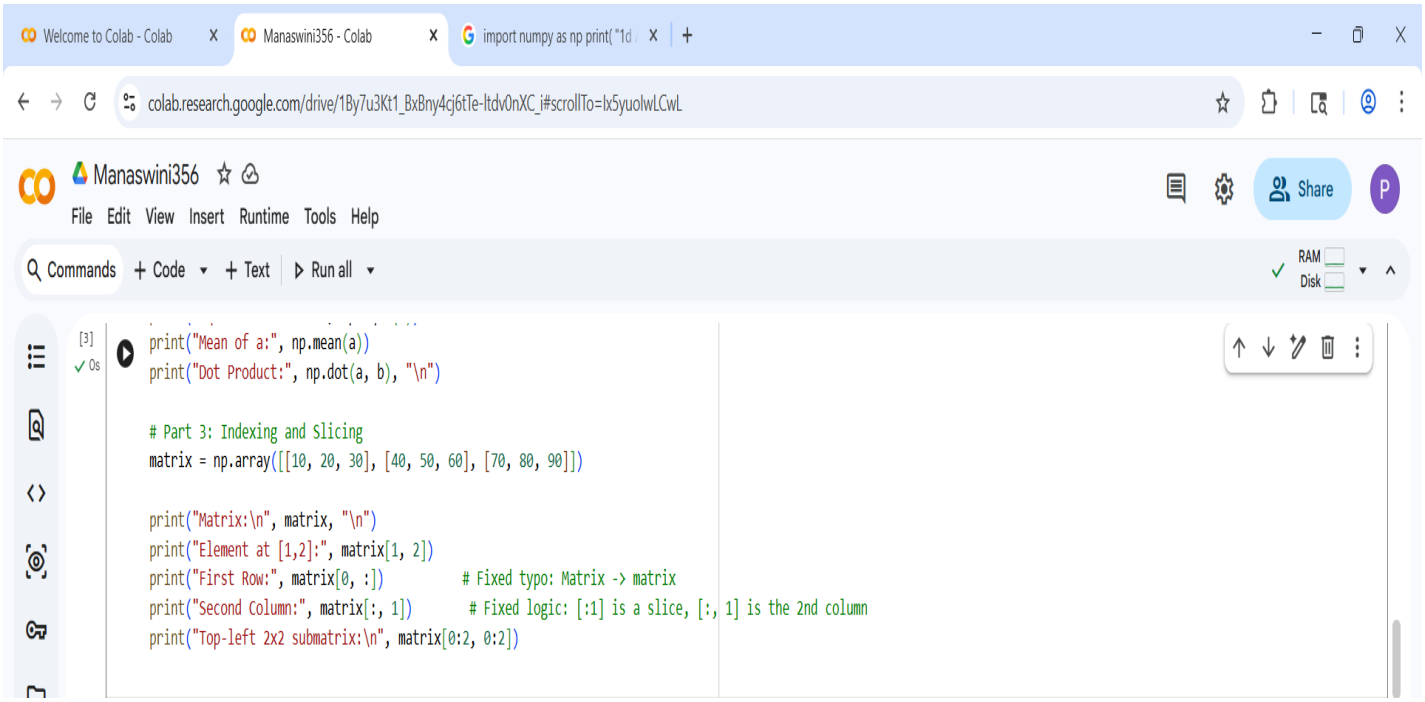
INDEX

EXP NO:1

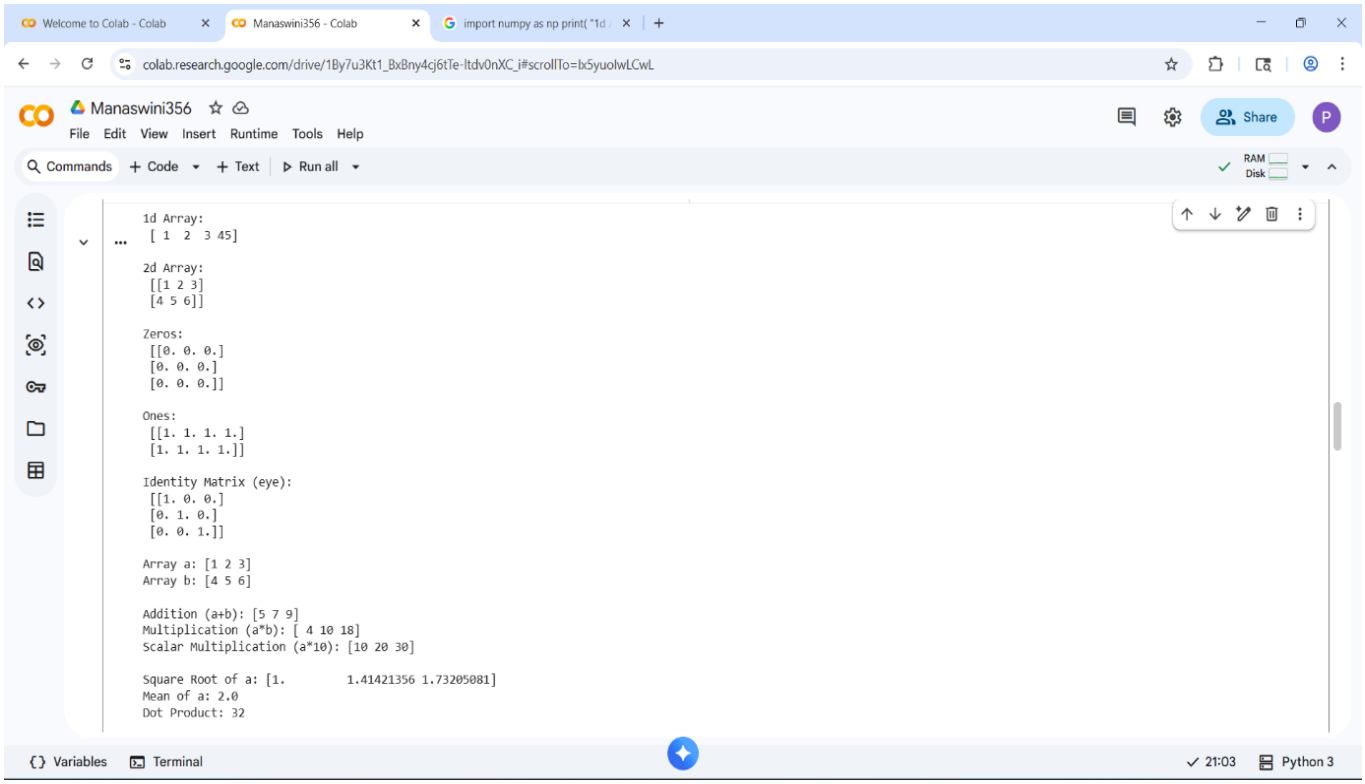
AIM: Create Arrays ,Array Operations ,Indexing and Slicing

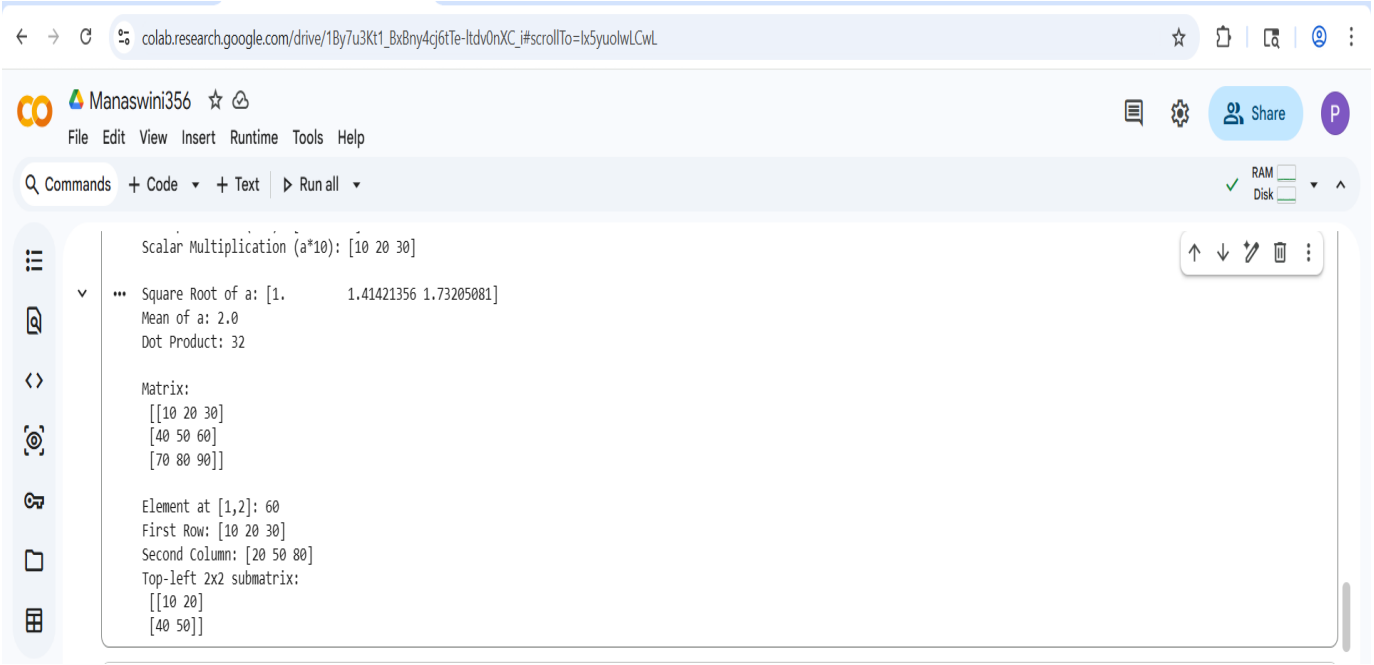
PROGRAM:





OUTPUT:





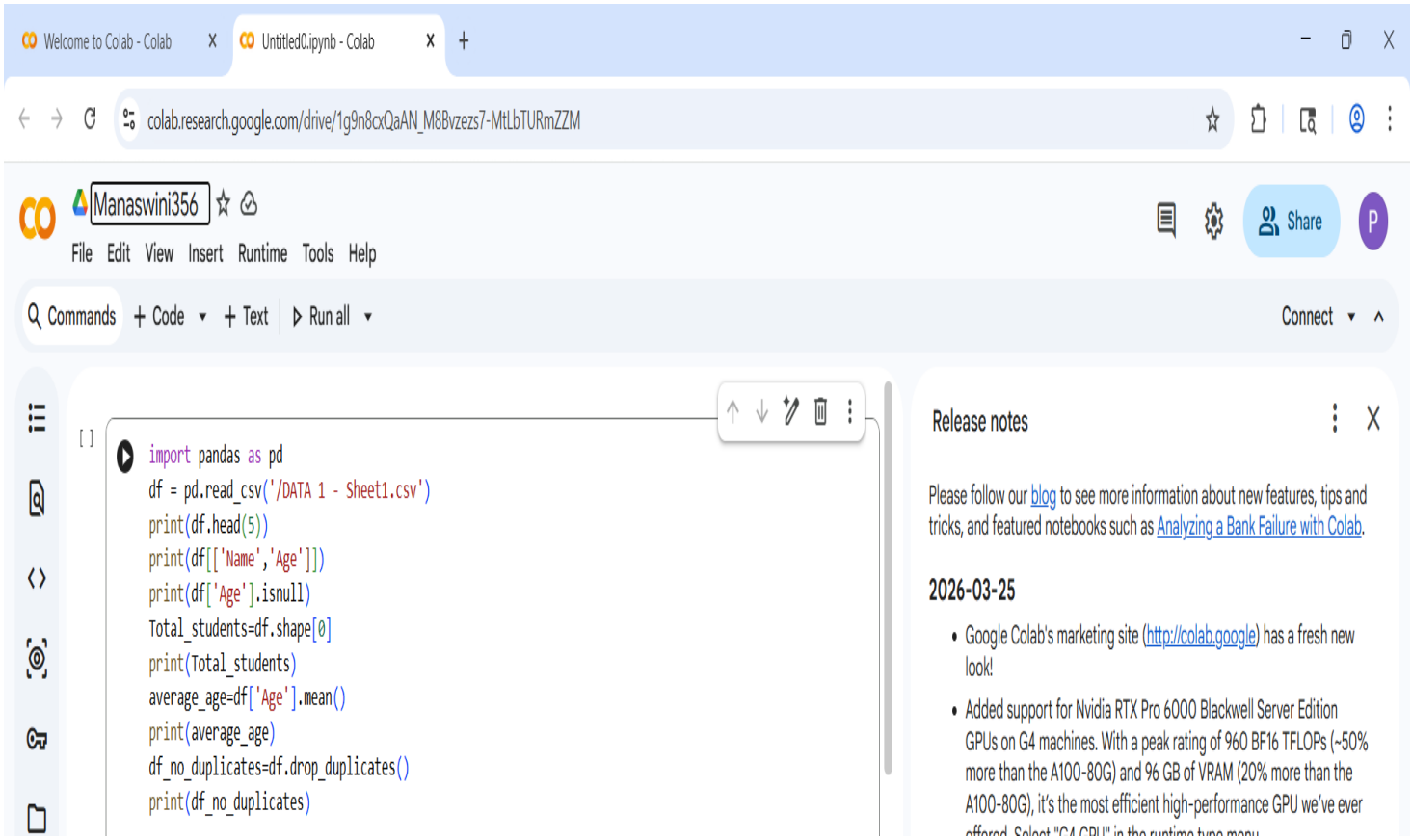
RESULT:

Basic NumPy operations were performed successfully

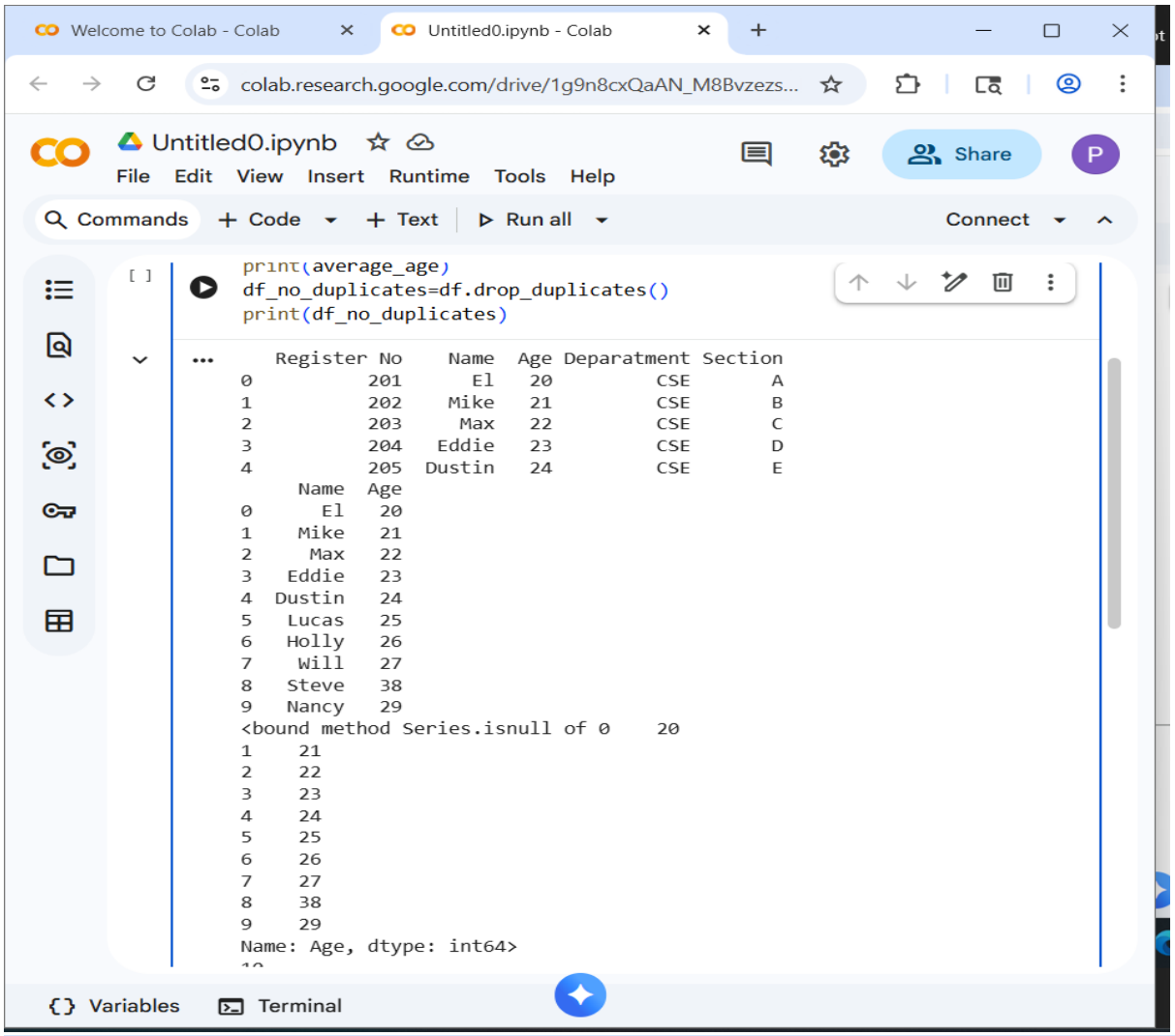
EXP: 1.1

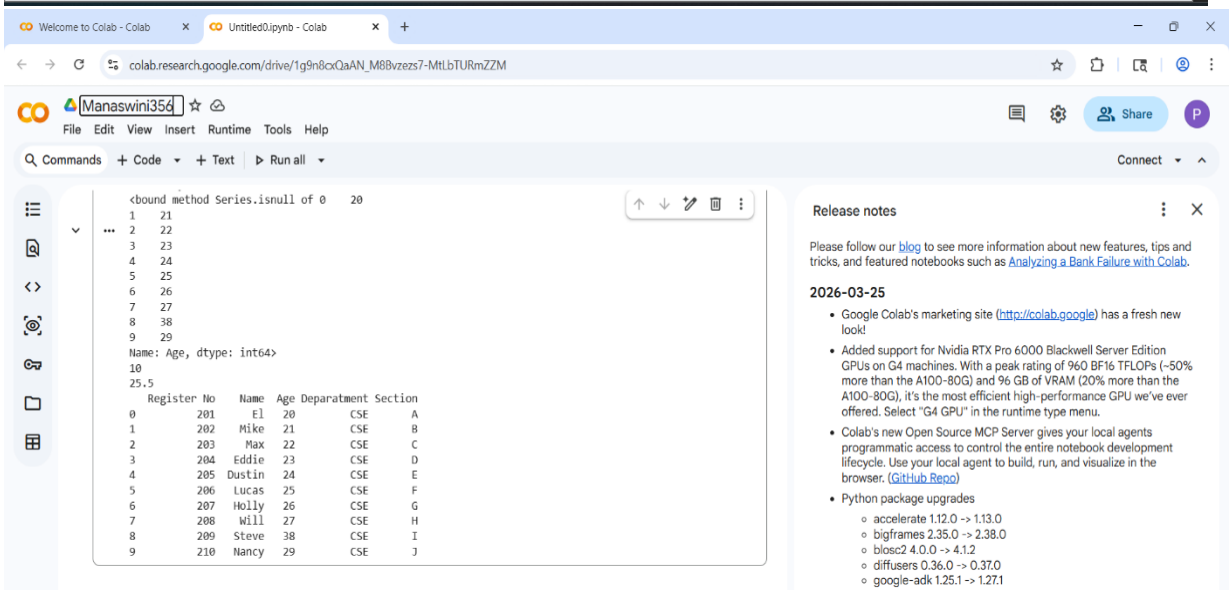
AIM: Creating Pandas Operations

PROGRAM:



OUTPUT:


Welcome to Colab - Colab
Untitled0.ipynb - Colab
colab.research.google.com/drive/1g9n8cxQaAN_M8Bvzezs...
Untitled0.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
Connect
[ ]
print(average_age)
df_no_duplicates=df.drop_duplicates()
print(df_no_duplicates)
Register No Name Age Deparatment Section
0 201 El 20 CSE A
1 202 Mike 21 CSE B
2 203 Max 22 CSE C
3 204 Eddie 23 CSE D
4 205 Dustin 24 CSE E
Name Age
0 El 20
1 Mike 21
2 Max 22
3 Eddie 23
4 Dustin 24
5 Lucas 25
6 Holly 26
7 Will 27
8 Steve 38
9 Nancy 29
<bound method Series.isnull of 0 20
1 21
2 22
3 23
4 24
5 25
6 26
7 27
8 38
9 29
Name: Age, dtype: int64>
Variables
Terminal

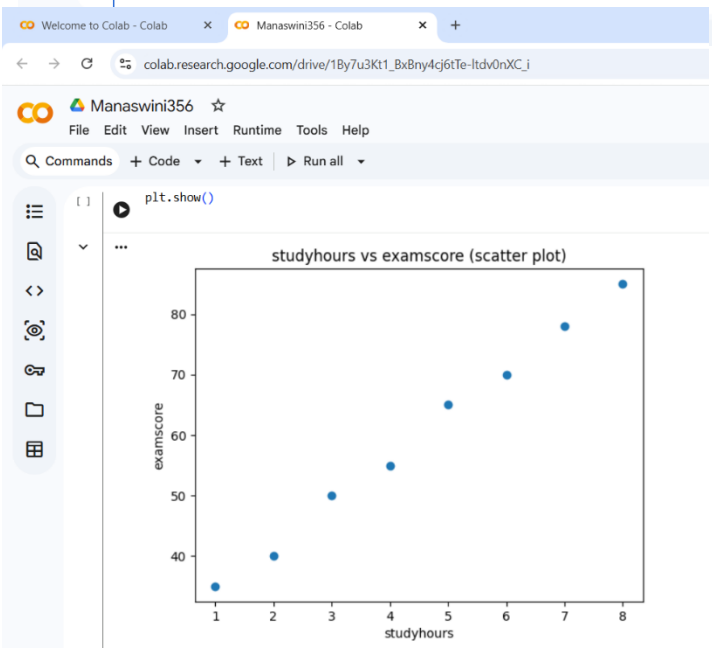

colab.research.google.com/drive/1g9n8cxQaAN_M8Bvzezs7-MtLbTURmZZM
Manaswini356
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
Connect
<bound method Series.isnull of 0 20
1 21
2 22
3 23
4 24
5 25
6 26
7 27
8 38
9 29
Name: Age, dtype: int64>
10
25.5
Register No Name Age Deparatment Section
0 201 El 20 CSE A
1 202 Mike 21 CSE B
2 203 Max 22 CSE C
3 204 Eddie 23 CSE D
4 205 Dustin 24 CSE E
5 206 Lucas 25 CSE F
6 207 Holly 26 CSE G
7 208 Will 27 CSE H
8 209 Steve 38 CSE I
9 210 Nancy 29 CSE J
Release notes
Please follow our blog to see more information about new features, tips and tricks, and featured notebooks such as Analyzing a Bank Failure with Colab.
2026-03-25
Google Colab's marketing site (http://colab.google) has a fresh new look!
Added support for Nvidia RTX Pro 6000 Blackwell Server Edition GPUs on G4 machines. With a peak rating of 960 BF16 TFLOPs (~50% more than the A100-80G) and 96 GB of VRAM (20% more than the A100-80G), it's the most efficient high-performance GPU we've ever offered. Select "G4 GPU" in the runtime type menu.
Colab's new Open Source MCP Server gives your local agents programmatic access to control the entire notebook development lifecycle. Use your local agent to build, run, and visualize in the browser. (GitHub Repo)
Python package upgrades
accelerate 1.12.0 -> 1.13.0
bigframes 2.35.0 -> 2.38.0
blosc2 4.0.0 -> 4.1.2
diffusers 0.36.0 -> 0.37.0
google-adk 1.25.1 -> 1.27.1

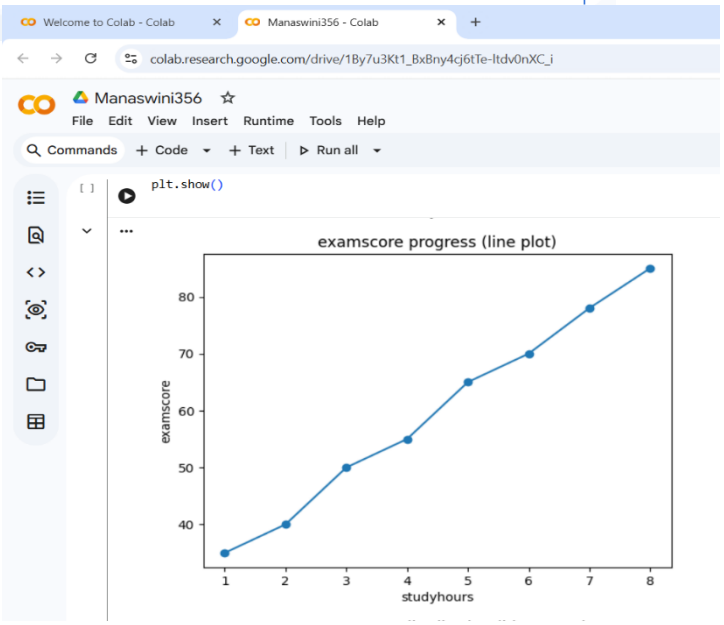
EXP: 2

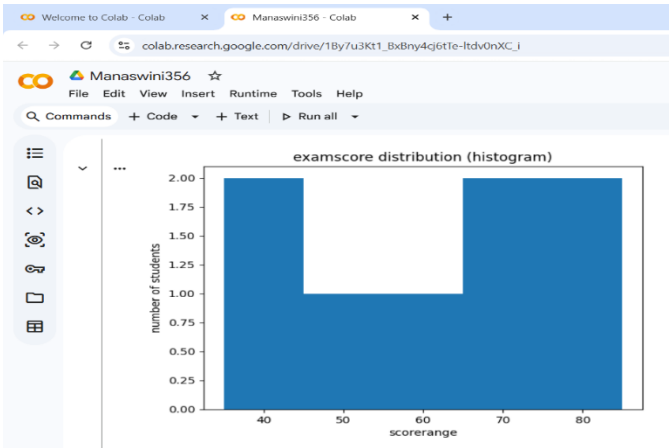
AIM: Creating Matplotlib and ScikitLearn &Data exploration & Preprocessing in Python

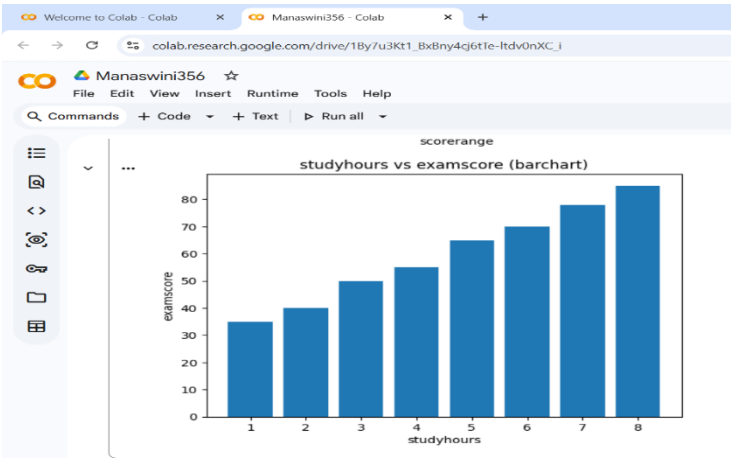
PROGRAM:

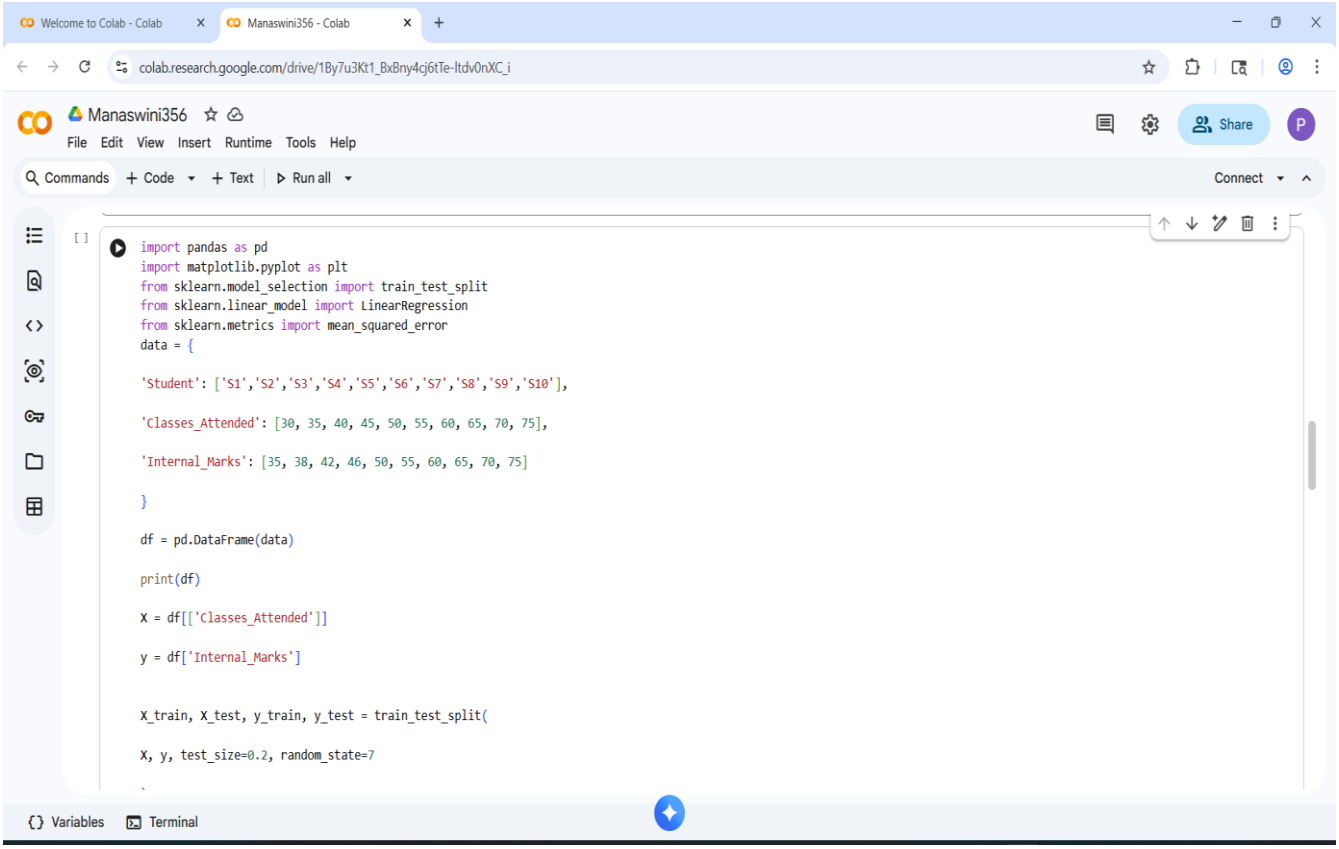
```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
data={
    'studyhours':[1,2,3,4,5,6,7,8],
    'examscore':[35,40,50,55,65,70,78,85]
}
df=pd.DataFrame(data)
plt.scatter(df['studyhours'],df['examscore'])
plt.xlabel("studyhours")
plt.ylabel("examscore")
plt.title("studyhours vs examscore (scatter plot)")
plt.show()
plt.plot(df['studyhours'],df['examscore'],marker='o')
plt.xlabel("studyhours")
plt.ylabel("examscore")
plt.title("examscore progress (line plot)")
plt.show()
plt.hist(df['examscore'],bins=5)
plt.xlabel("scorerange")
plt.ylabel("number of students")
plt.title("examscore distribution (histogram)")
plt.show()
plt.bar(df['studyhours'],df['examscore'])
plt.xlabel("studyhours")
plt.ylabel("examscore")
plt.title("studyhours vs examscore (barchart)")
plt.show()
```

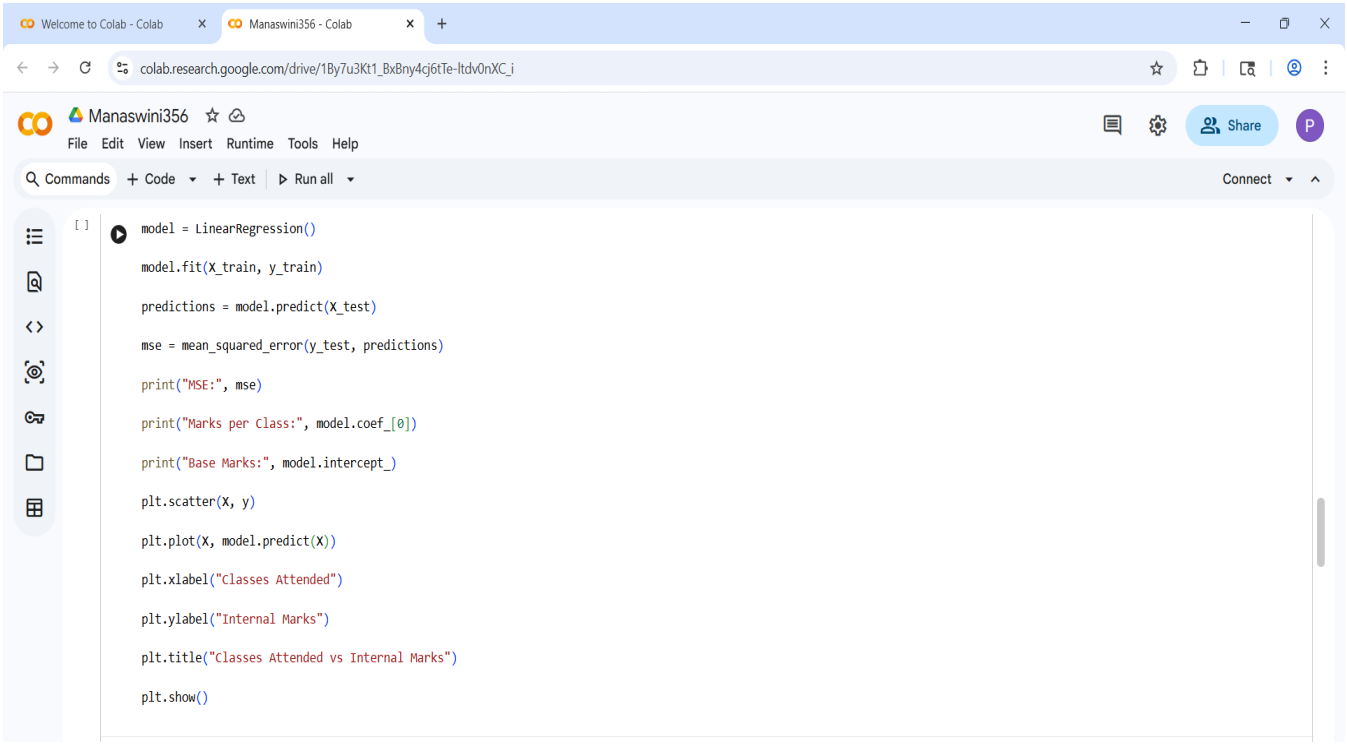




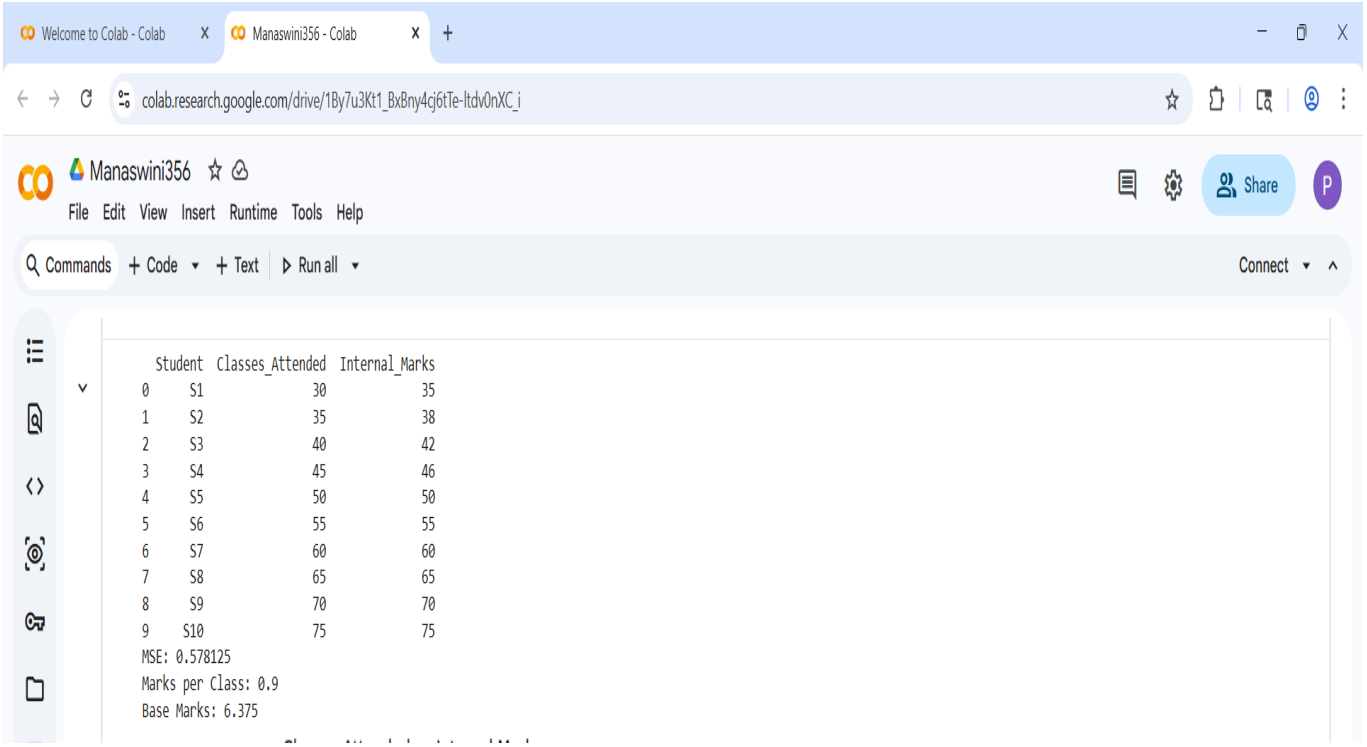


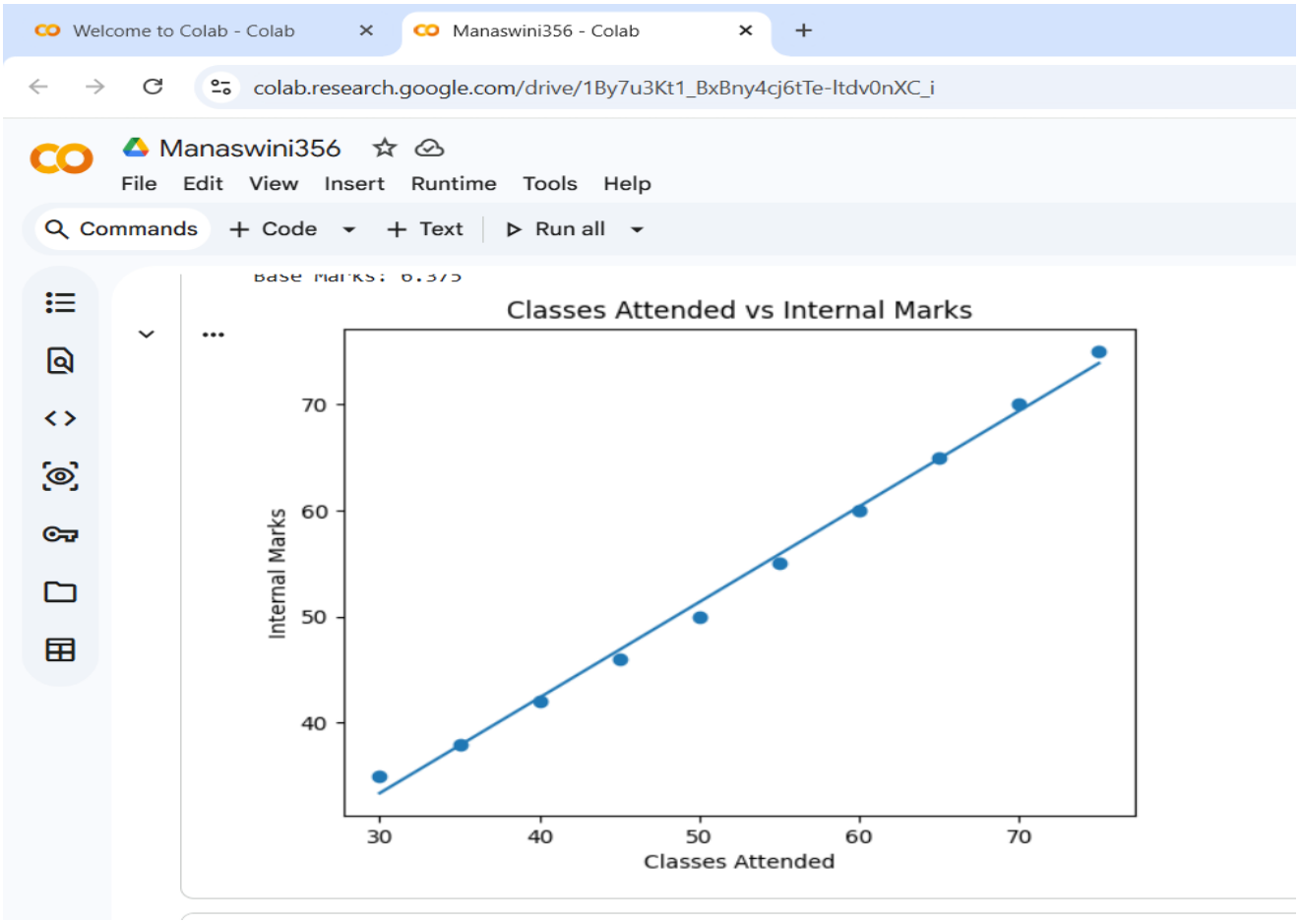



Welcome to Colab - Colab
Manaswini356 - Colab
colab.research.google.com/drive/1By7u3Kt1_BxBny4cj6tTe-ltdv0nXC_i
Manaswini356
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
Connect
[ ]
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
data = {
'Student': ['S1','S2','S3','S4','S5','S6','S7','S8','S9','S10'],
'Classes_Attended': [30, 35, 40, 45, 50, 55, 60, 65, 70, 75],
'Internal_Marks': [35, 38, 42, 46, 50, 55, 60, 65, 70, 75]
}
df = pd.DataFrame(data)
print(df)
X = df[['Classes_Attended']]
y = df['Internal_Marks']
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=7
Variables
Terminal


Welcome to Colab - Colab
Manaswini356 - Colab
colab.research.google.com/drive/1By7u3Kt1_BxBny4cj6tTe-ltdv0nXC_i
Manaswini356
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
Connect
[ ]
model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
mse = mean_squared_error(y_test, predictions)
print("MSE:", mse)
print("Marks per Class:", model.coef_[0])
print("Base Marks:", model.intercept_)
plt.scatter(X, y)
plt.plot(X, model.predict(X))
plt.xlabel("Classes Attended")
plt.ylabel("Internal Marks")
plt.title("Classes Attended vs Internal Marks")
plt.show()

OUTPUT:

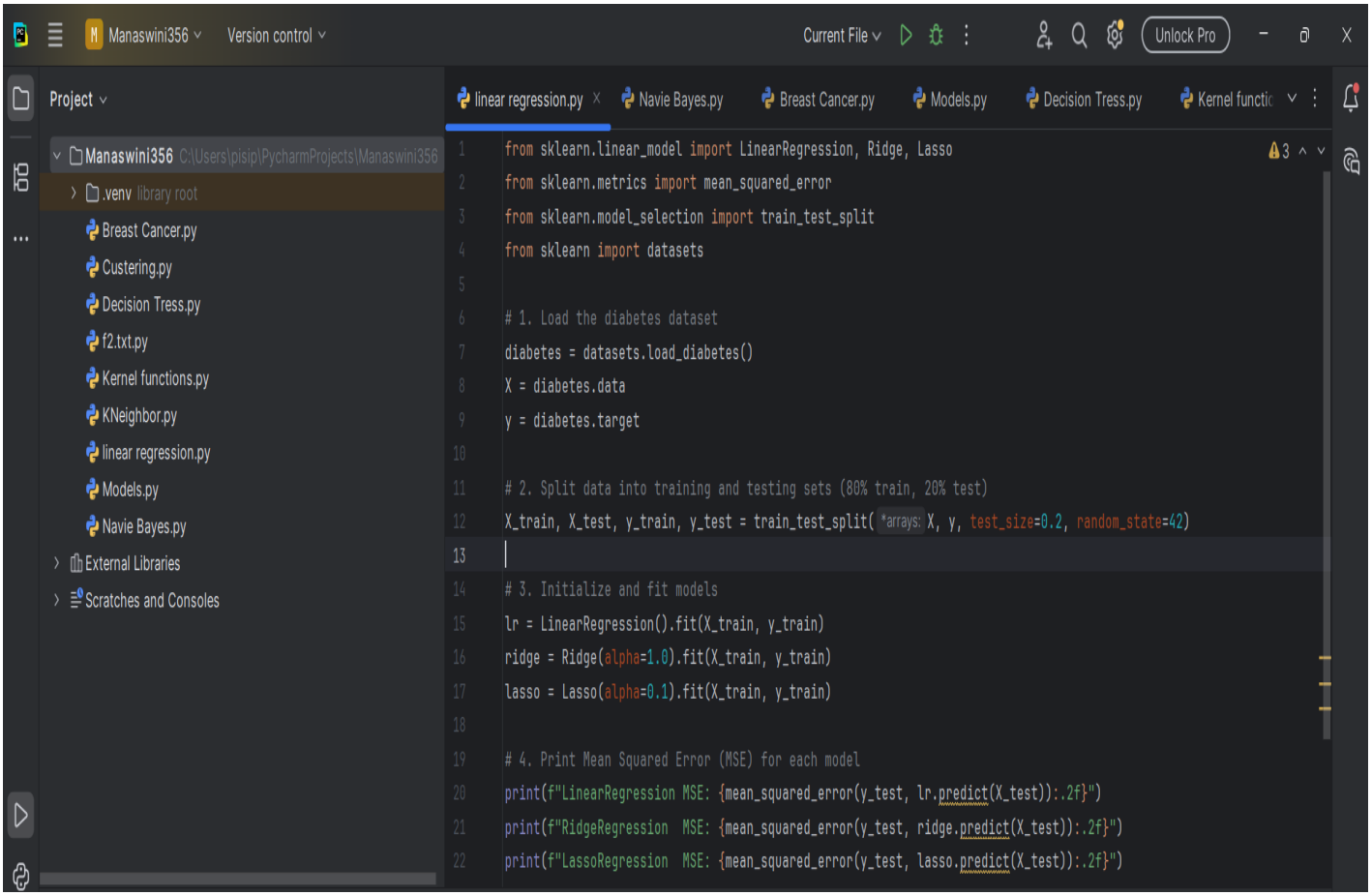

Welcome to Colab - Colab
Manaswini356 - Colab
colab.research.google.com/drive/1By7u3Kt1_BxBny4cj6tTe-ltdv0nXC_i
Manaswini356
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Connect
Share
Student Classes_Attended Internal_Marks
0 S1 30 35
1 S2 35 38
2 S3 40 42
3 S4 45 46
4 S5 50 50
5 S6 55 55
6 S7 60 60
7 S8 65 65
8 S9 70 70
9 S10 75 75
MSE: 0.578125
Marks per Class: 0.9
Base Marks: 6.375


Welcome to Colab - Colab
Manaswini356 - Colab
colab.research.google.com/drive/1By7u3Kt1_BxBny4cj6tTe-ltdv0nXC_i
Manaswini356
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Classes Attended vs Internal Marks
Internal Marks
Classes Attended
70
60
50
40
30
40
50
60
70

EXP: 3

AIM: Implement Regularized Linear Regression

PROGRAM:



OUTPUT:

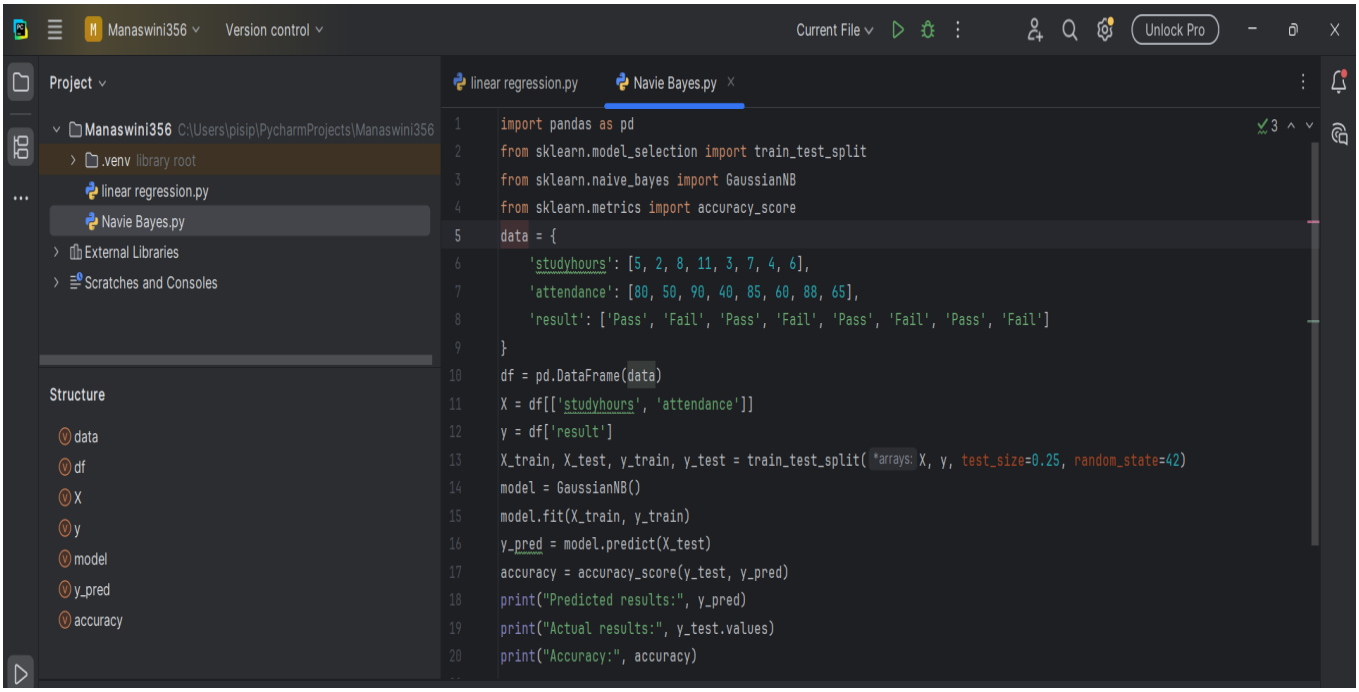
RESULT:

The model was successfully implemented and regularization heiped reduce overfitting and imporved prediction accuracy.

EXP: 4

AIM: Implement Navie Bayes Classifer

PROGRAM:



OUTPUT:

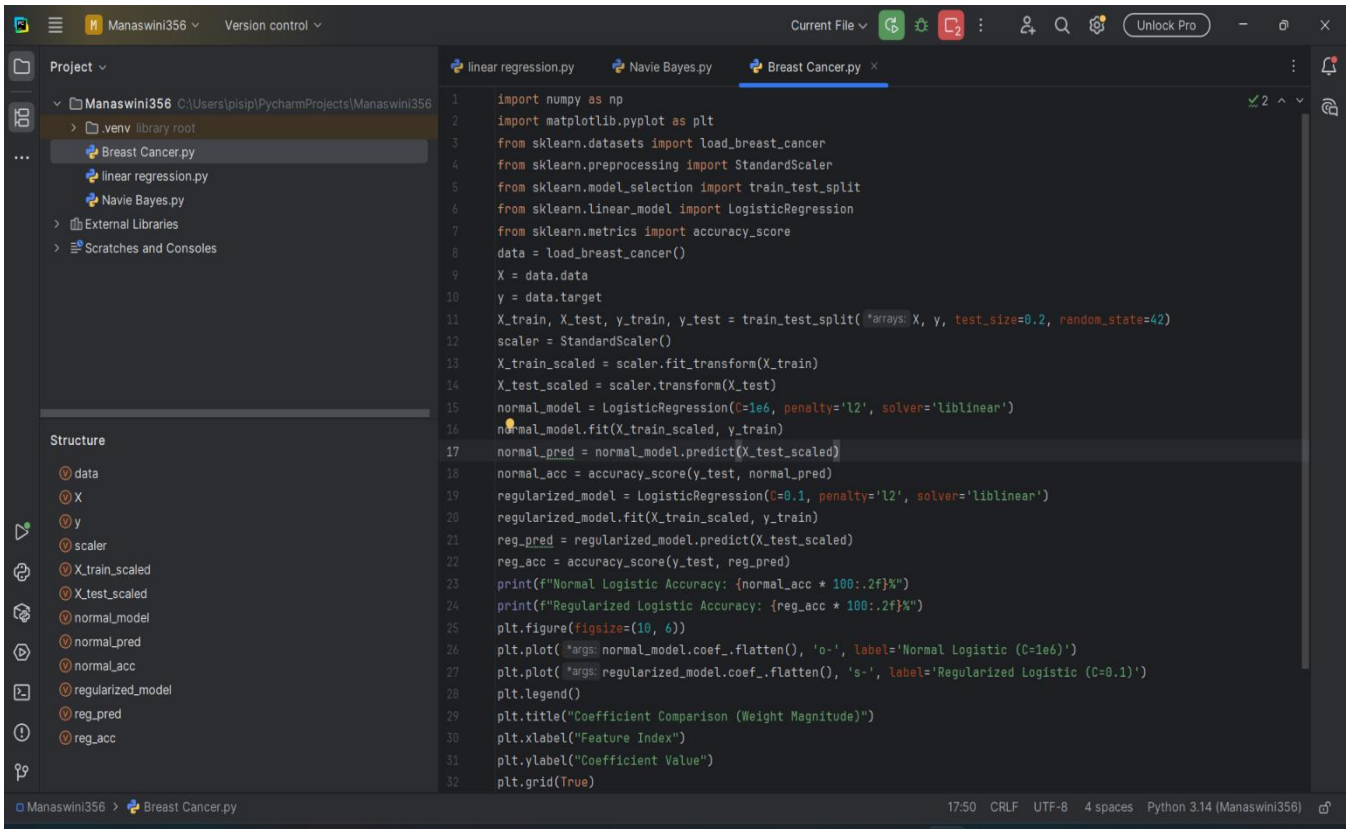
RESULT:

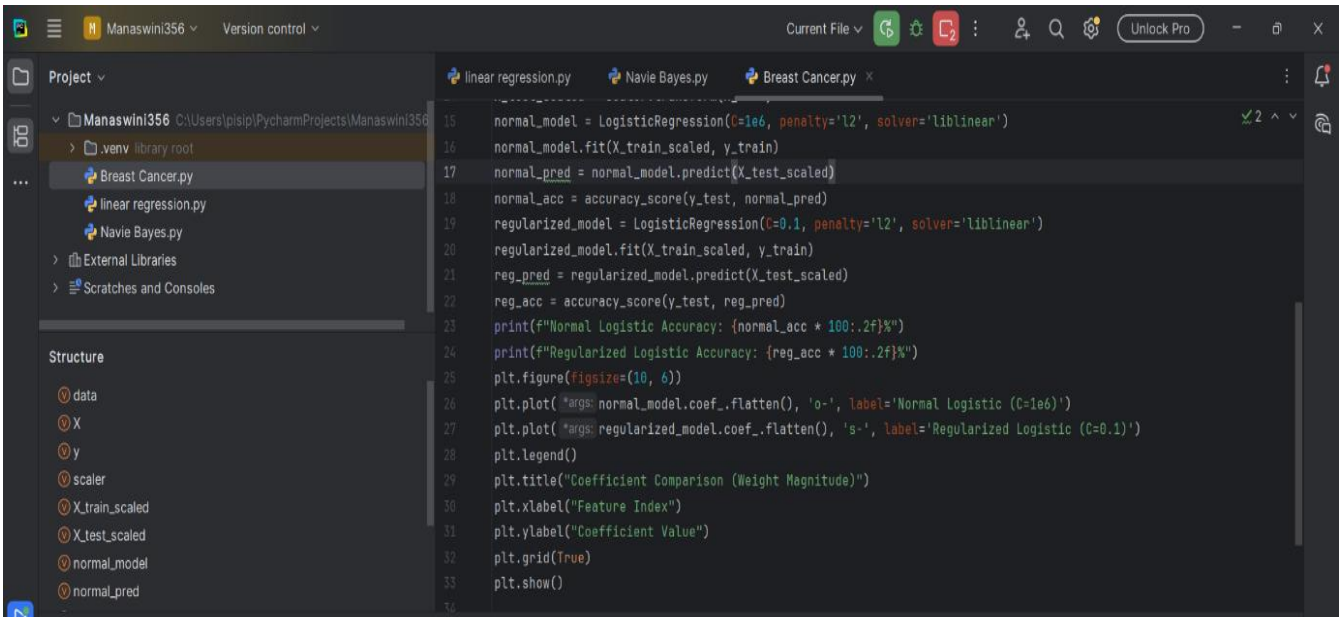
The Navie Bayes effectively classified the data with good accuracy.

EXP: 5

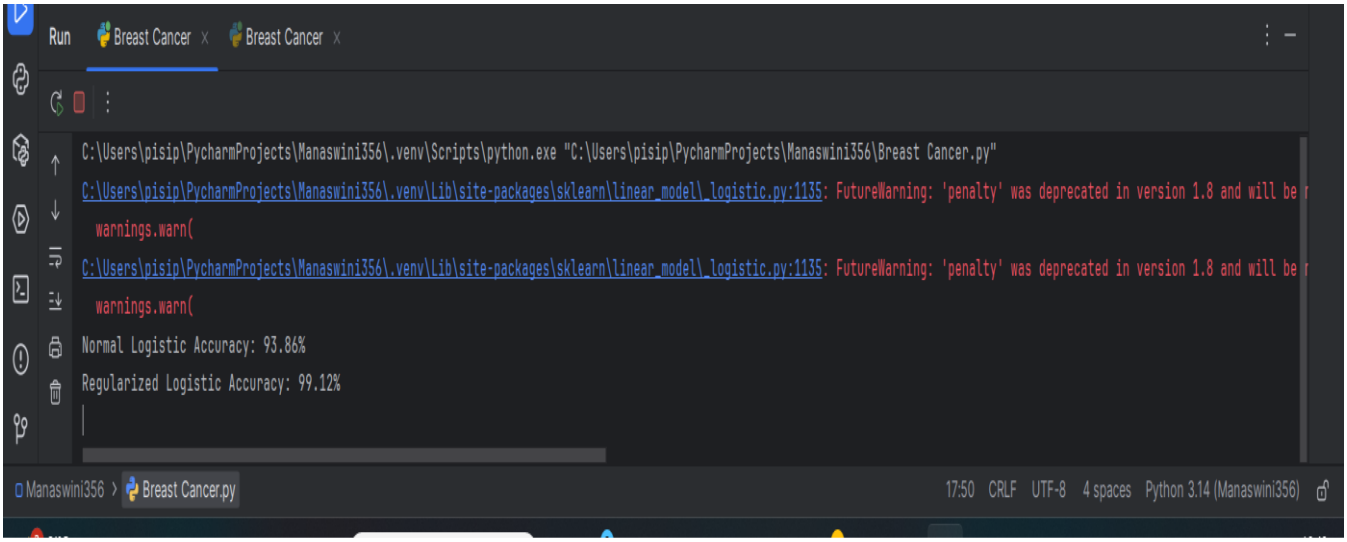
AIM: Implement Regularized Logistic Regression

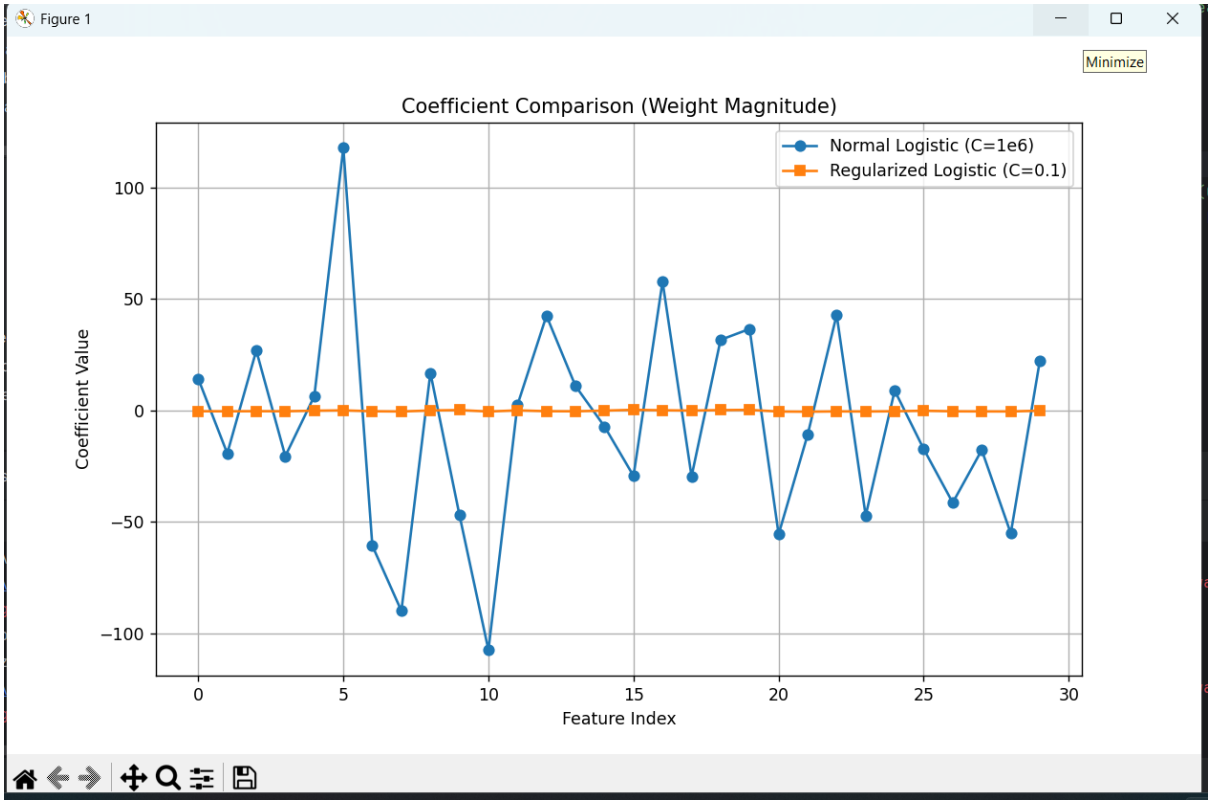
PROGRAM:





OUTPUT:





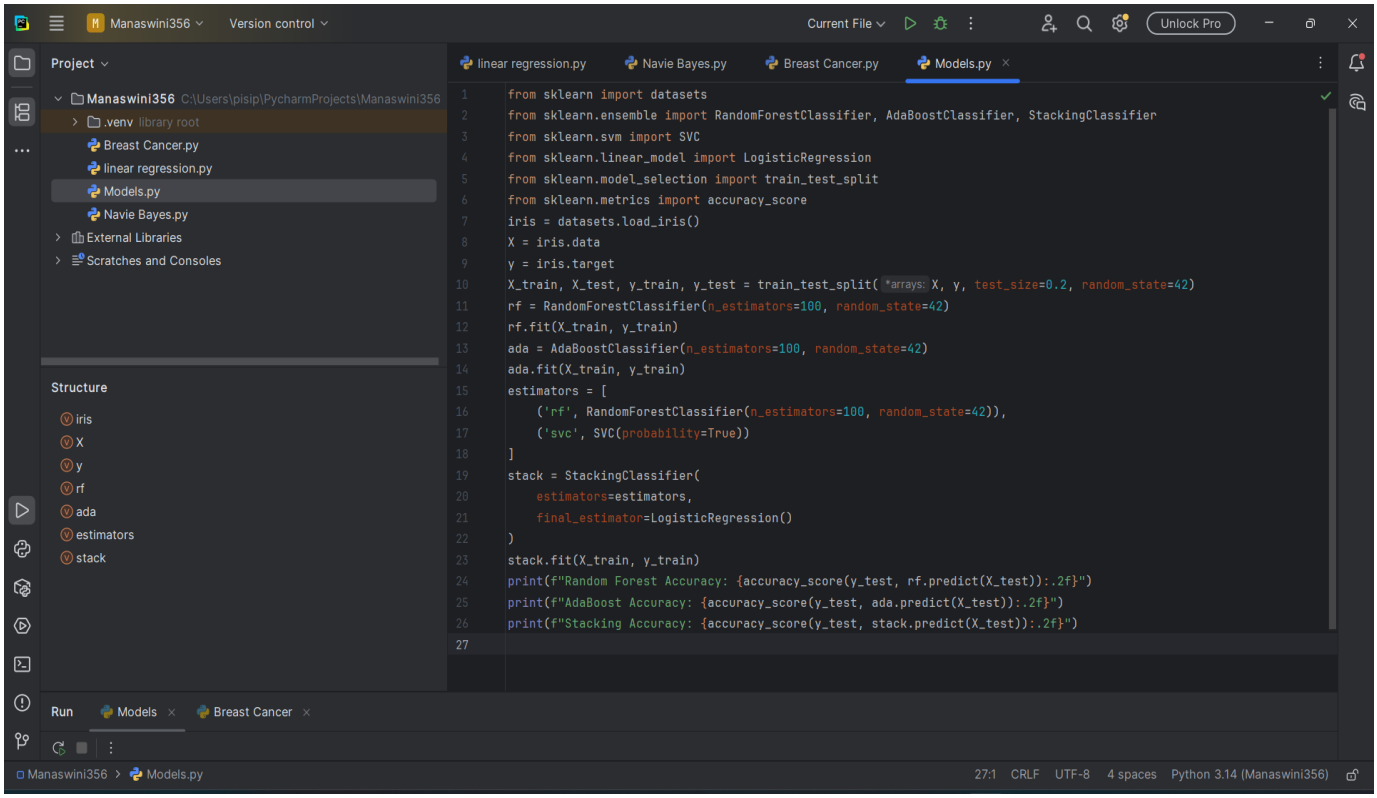
RESULT:

The model was successfully implemented and regularization imporved accuracy and prevented overfitting.

EXP: 6

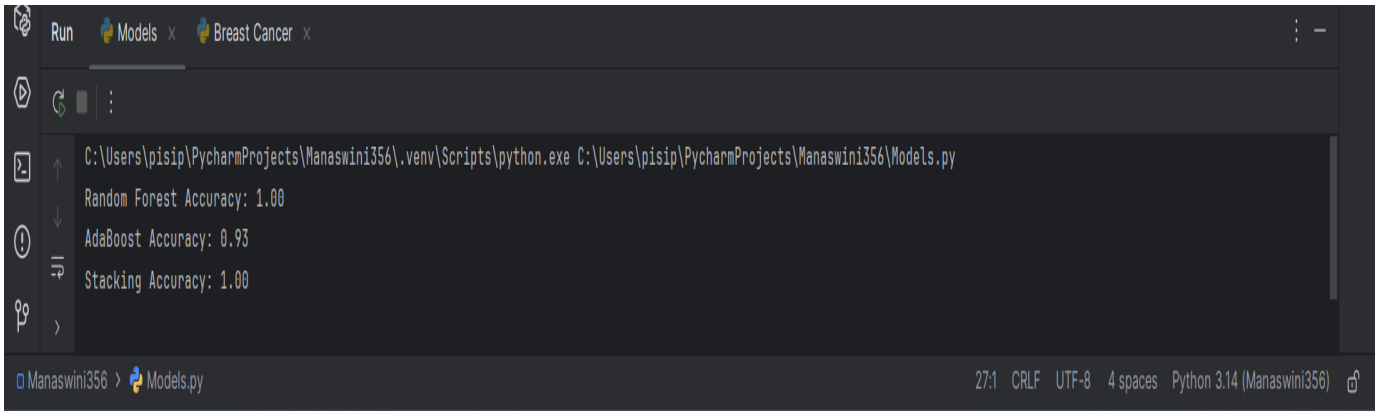
AIM: To Build Models using different Ensembling Techniques

PROGRAM:



```
from sklearn import datasets
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, StackingClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
iris = datasets.load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split( *arrays: X, y, test_size=0.2, random_state=42)
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
ada = AdaBoostClassifier(n_estimators=100, random_state=42)
ada.fit(X_train, y_train)
estimators = [
    ('rf', RandomForestClassifier(n_estimators=100, random_state=42)),
    ('svc', SVC(probability=True))
]
stack = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression()
)
stack.fit(X_train, y_train)
print(f"Random Forest Accuracy: {accuracy_score(y_test, rf.predict(X_test)):.2f}")
print(f"AdaBoost Accuracy: {accuracy_score(y_test, ada.predict(X_test)):.2f}")
print(f"Stacking Accuracy: {accuracy_score(y_test, stack.predict(X_test)):.2f}")
```

OUTPUT:



```
C:\Users\pisip\PycharmProjects\Manaswini356\.venv\Scripts\python.exe C:\Users\pisip\PycharmProjects\Manaswini356\Models.py
Random Forest Accuracy: 1.00
AdaBoost Accuracy: 0.93
Stacking Accuracy: 1.00
```

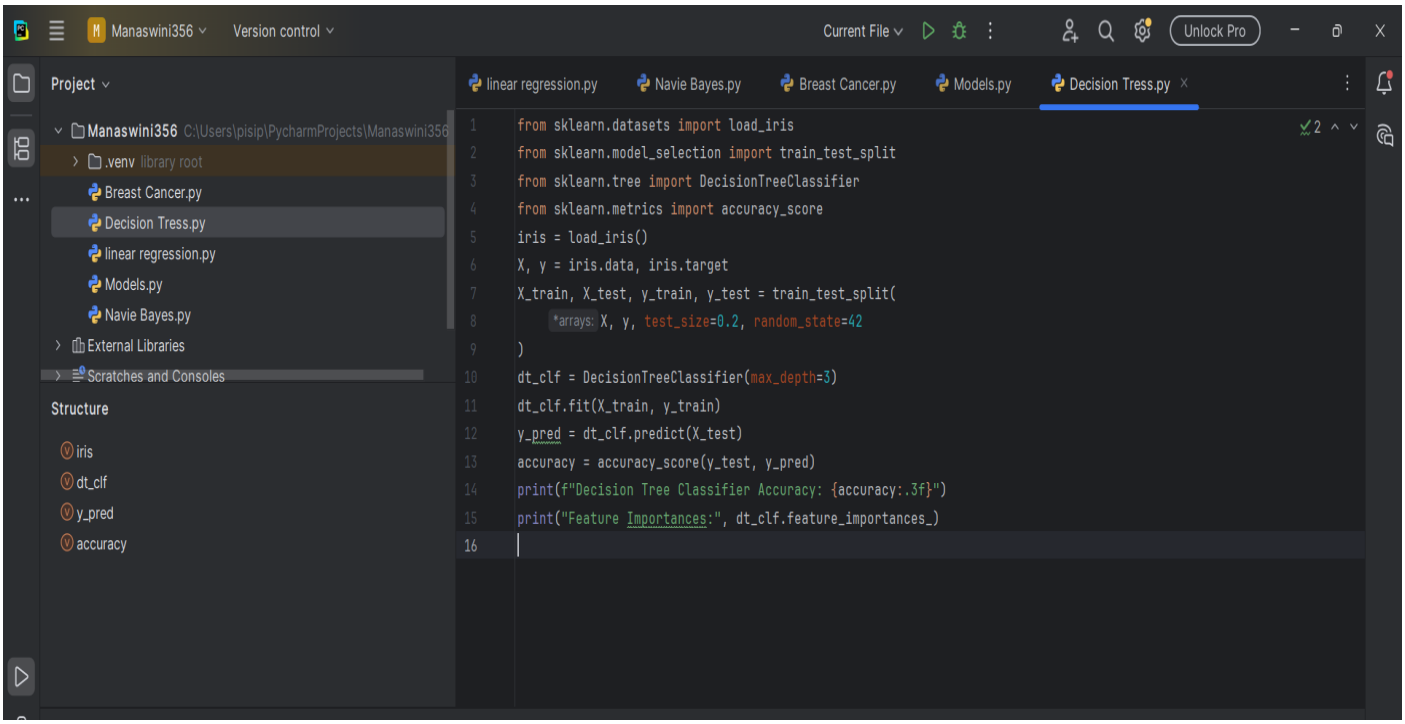
RESULT:

The ensemble models were successfully implemented and they provided better accuracy compared to indival models.

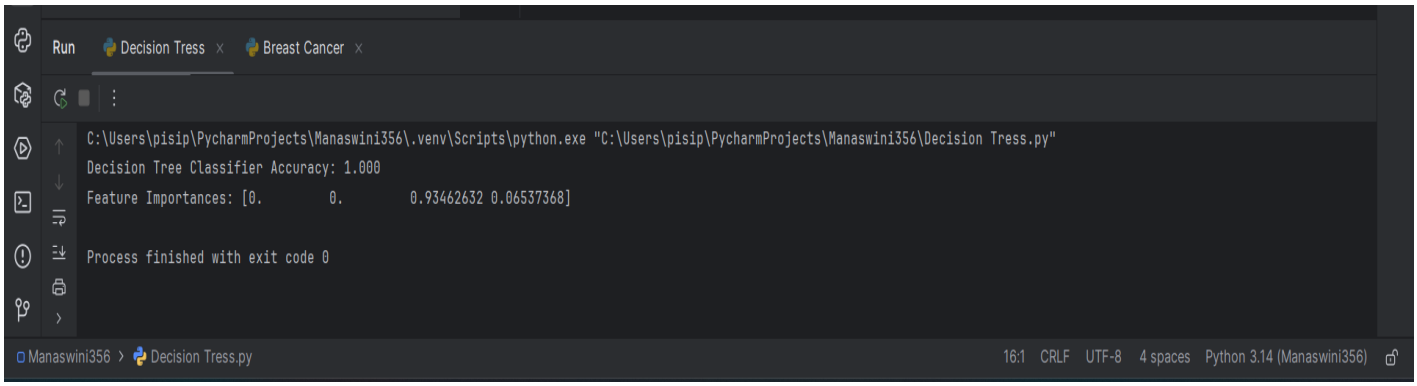
EXP: 7

AIM: To Build Models using Decision Tress

POGRAM:



OUTPUT:



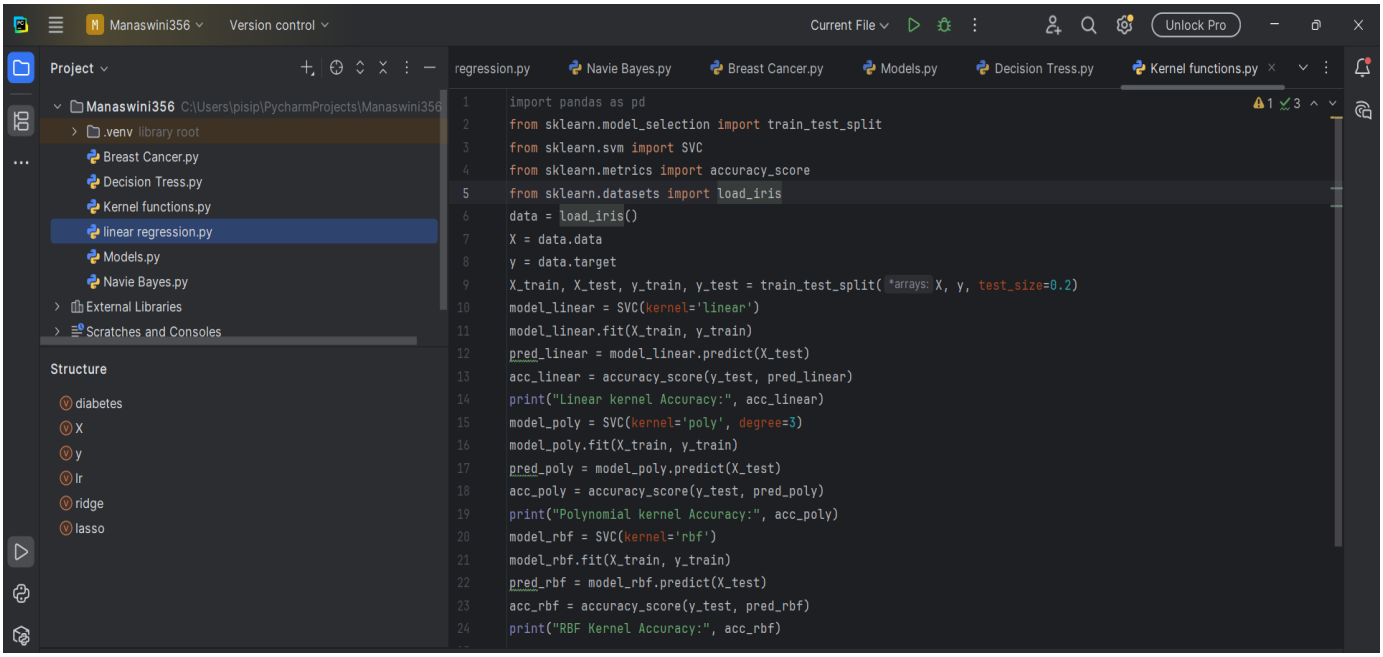
RESULT:

The Decision Tree model was successfully implemented and provided good prediction performance.

EXP: 8

AIM: Build model using SVM with different Kernels

PROGRAM:



```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
data = load_iris()
X = data.data
y = data.target
X_train, X_test, y_train, y_test = train_test_split( *arrays: X, y, test_size=0.2)
model_linear = SVC(kernel='linear')
model_linear.fit(X_train, y_train)
pred_linear = model_linear.predict(X_test)
acc_linear = accuracy_score(y_test, pred_linear)
print("Linear kernel Accuracy:", acc_linear)
model_poly = SVC(kernel='poly', degree=3)
model_poly.fit(X_train, y_train)
pred_poly = model_poly.predict(X_test)
acc_poly = accuracy_score(y_test, pred_poly)
print("Polynomial kernel Accuracy:", acc_poly)
model_rbf = SVC(kernel='rbf')
model_rbf.fit(X_train, y_train)
pred_rbf = model_rbf.predict(X_test)
acc_rbf = accuracy_score(y_test, pred_rbf)
print("RBF Kernel Accuracy:", acc_rbf)
```

OUTPUT:

```
C:\Users\pisip\PycharmProjects\Manaswini356\.venv\Scripts\python.exe "C:\Users\pisip\PycharmProjects\Manaswini356\Kernel functions.py"
Linear kernel Accuracy: 1.0
Polynomial kernel Accuracy: 0.9666666666666667
RBF Kernel Accuracy: 0.9666666666666667
```

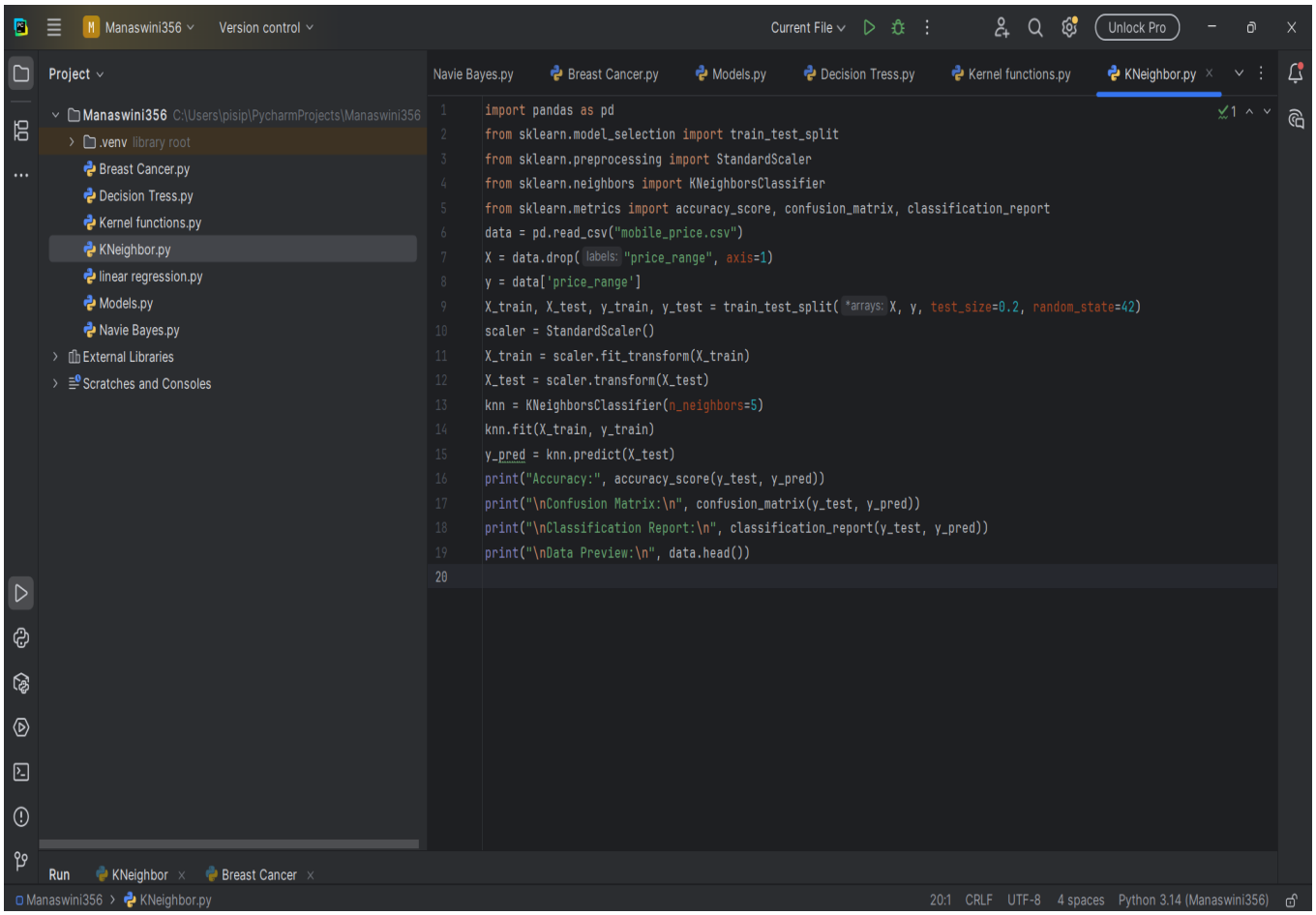
RESULT:

The SVM models were successfully implemented with various kernels and they achived good classification performance.

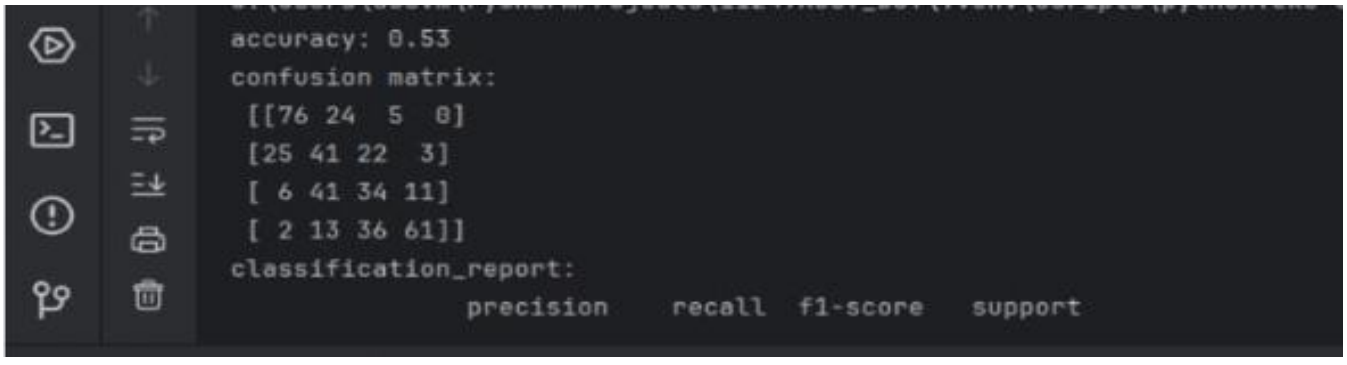
EXP: 9

AIM: Implement K-NN algorithm to classify a dataset

PROGRAM:



OUTPUT:



RESULT:

The K-NN model was successfully implemented and it is classified.

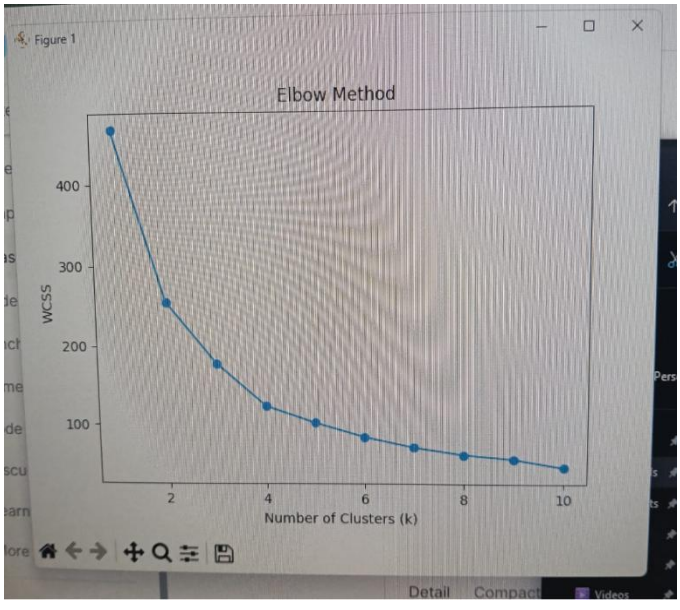
EXP: 10

AIM: To build a K-Means Clustering with PCA and Elbow method

PROGRAM:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
data = pd.read_csv("Mall_Customers(1).csv")
X = data[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("Explained variance ratio:", pca.explained_variance_ratio_)
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=0, n_init=10)
    kmeans.fit(X_pca)
    wcss.append(kmeans.inertia_)
plt.figure(figsize=(8, 5))
plt.plot( *args: range(1, 11), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.show()
kmeans = KMeans(n_clusters=5, random_state=0, n_init=10)
clusters = kmeans.fit_predict(X_pca)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, cmap='viridis')
plt.title("K-means Clustering with PCA")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.show()
```

OUTPUT:



RESULT:

The K-Means Clustering was successfully implemented after PCA and Elbow method and the data was grouped effectively into clusters.